

Trabalho Prático 01 - Algoritmos 2

Bruno Lopes Melo Fonseca - 2023087869

Gabriel Alkmim Barros - 2023106049

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte - MG - Brasil

brunolmf@ufmg.br
gabrielalkmim6@ufmg.br

Introdução do Problema

Neste trabalho prático temos como objetivo desenvolver um buscador de notícias a partir da implementação de algoritmos e estruturas de dados voltados à manipulação de sequências e recuperação de informação. O projeto busca aplicar, de forma prática, os conceitos teóricos estudados em sala de aula, promovendo uma compreensão mais profunda sobre o funcionamento de mecanismos de busca e o uso eficiente de estruturas de dados.

O sistema será dividido em três módulos principais:

- Indexação, responsável por processar os documentos e construir um índice invertido utilizando uma árvore Trie compacta, estrutura que associa cada termo aos documentos em que ocorre;
- Recuperação de Informação (RI), responsável por interpretar consultas booleanas, combinar operadores lógicos (AND, OR) e calcular a relevância dos documentos por meio da média dos z-scores dos termos pesquisados;
- Interface com o Usuário, desenvolvida com o framework Flask, que exibirá os resultados em páginas organizadas e apresentará trechos (snippets) dos textos contendo os termos mais relevantes destacados.

Decisões de Projeto

Esta seção descreve o processo de implementação do sistema de indexação e recuperação de informações textuais, detalhando as principais decisões de projeto, as estruturas de dados adotadas e os mecanismos de funcionamento de cada módulo. O sistema foi desenvolvido em **Python**, segundo o que pede a documentação, oferecendo alta legibilidade, boa integração com expressões regulares e manipulação simples de arquivos texto. A estrutura central é uma *Trie Compacta*, utilizada para armazenar e recuperar termos de forma eficiente, complementada por módulos responsáveis pela indexação, consultas booleanas e ranqueamento por relevância.

0.1 Estrutura da Trie Compacta

A **TrieCompacta** é uma variação otimizada da estrutura de árvore prefixada tradicional (*trie*). Nessa implementação, caminhos formados por nós intermediários sem ramificações são condensados

em uma única aresta, reduzindo significativamente o número de nós e o consumo de memória, sem comprometer a capacidade de busca.

Cada nó da estrutura, representado pela classe `TrieNode`, contém três componentes principais:

- **children**: dicionário que associa rótulos de arestas (substrings) aos nós-filhos correspondentes;
- **postings**: dicionário que mapeia identificadores de documentos (`doc_id`) para suas frequências de ocorrência do termo;
- **is_end**: flag booleana que indica se o caminho até o nó representa um termo completo.

A classe `TrieCompacta` implementa as operações fundamentais:

- **insert(word, doc_id)**: insere um termo na estrutura, tratando automaticamente os três casos possíveis:
 1. o prefixo do termo coincide totalmente com uma aresta existente (continua-se a inserção);
 2. há coincidência parcial, exigindo divisão da aresta por meio do método interno `_split_edge()`;
 3. não há prefixo comum, criando-se uma nova aresta e nó terminal.
- **search(word)**: busca um termo e retorna o conjunto de identificadores de documentos que o contêm.
- **get_all_words_with_postings()**: percorre toda a estrutura e retorna todas as palavras indexadas e suas listas de postings.

A adoção da *trie compacta*, em vez da versão simples, foi uma decisão motivada pela necessidade de otimizar o armazenamento e a velocidade de acesso, especialmente relevante em corpora extensos.

0.2 Módulo Indexador

O módulo `Indexador` é responsável pela construção e manutenção do índice invertido. Ele percorre o corpus de textos, extrai os termos e os insere na `TrieCompacta`, além de oferecer mecanismos de persistência e reconstrução do índice a partir de arquivo.

Os principais atributos da classe `Indexador` são:

- **caminho_corpus**: diretório raiz contendo as categorias e documentos;
- **trie**: instância da estrutura compacta que armazena termos e frequências;
- **mapa_docs**: mapeamento de `doc_id` para caminhos de arquivos;
- **arquivo_indice**: arquivo texto onde o índice é salvo ou carregado.

Ao inicializar o objeto, o sistema verifica se já existe um índice salvo em disco. Caso positivo, ele é carregado; caso contrário, um novo índice é construído a partir do corpus. A tokenização dos textos é feita com expressões regulares (`re.findall`), extraindo apenas palavras alfabéticas e acentuadas. Todos os termos são convertidos para minúsculas, garantindo consistência durante as buscas.

Exemplo de log de indexação:

```
Indexando corpus/categoria1/1.txt (ID=1)...\nIndexando corpus/categoria1/2.txt (ID=2)...
```

Indexando corpus/categoria2/1.txt (ID=3)...

Indexação concluída com sucesso!

O método `salvar_indice()` grava o mapa de documentos e as postagens da trie em formato textual simples, permitindo inspeção direta e reconstrução posterior via `carregar_indice()`. Essa decisão buscou privilegiar transparência e portabilidade, evitando formatos binários.

0.3 Módulo de Recuperação da Informação

O módulo `RecuperacaoInformacao` é responsável pela interpretação de consultas, avaliação booleana e ranqueamento de resultados. Ele atua como camada superior do sistema, recebendo a instância do `Indexador` para acessar o índice e o mapeamento de documentos.

Árvore de Consulta e Operadores

Para representar a lógica das consultas, foi implementada a classe `NoConsulta`, que define nós contendo termos ou operadores (`AND`, `OR`). Esses nós são organizados em uma árvore sintática construída a partir da expressão textual do usuário.

O método `_construir_arvore()` analisa a sequência de tokens respeitando a precedência dos operadores — `AND` tem prioridade sobre `OR`. Se a consulta iniciar ou terminar com um operador lógico, o sistema a considera inválida, evitando erros semânticos. Da mesma forma, na ausência de operadores explícitos entre termos consecutivos, o sistema assume um `OR` implícito, permitindo consultas mais naturais.

Exemplo de consulta:

Entrada do usuário: vida amor esperança
Interpretação: vida OR amor OR esperança

Exemplo de árvore gerada:

Consulta: (vida AND amor) OR esperança

```
      OR
     /  \
    AND  esperança
   /    \
  vida   amor
```

Avaliação e Recuperação

Cada nó da árvore é avaliado recursivamente:

- Nós folha (termos) invocam `trie.search()`, retornando os documentos que contêm o termo;
- Nós internos aplicam `AND` (interseção) ou `OR` (união) sobre os conjuntos retornados.

Ranqueamento de Relevância

Para ordenar os resultados, foi implementado um ranqueamento baseado em *z-scores*, que mede o desvio de frequência de um termo em relação à média do corpus. Cada documento recebe um valor normalizado de relevância:

$$z = \frac{f - \mu}{\sigma}$$

onde f é a frequência do termo, μ a média das frequências e σ o desvio padrão. Os escores de todos os termos da consulta são combinados por média, e os documentos são ordenados em ordem decrescente de relevância.

Exemplo de saída:

Consulta: (vida AND amor) OR esperança

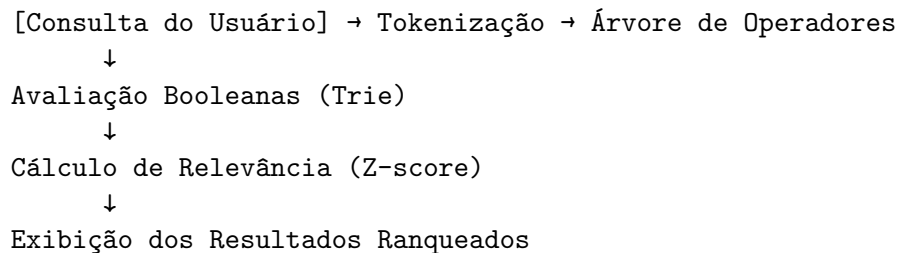
Documentos encontrados (3):

- corpus/categoria1/1.txt (relevância = 1.224)
- corpus/categoria1/2.txt (relevância = 0.512)
- corpus/categoria2/1.txt (relevância = 0.103)

0.4 Integração e Fluxo Geral

Os módulos funcionam de maneira encadeada: o **Indexador** constrói a base de dados, a **TrieCompacta** armazena eficientemente os termos e o módulo de **RecuperacaoInformacao** realiza consultas e ranqueia os resultados.

Fluxo resumido de execução:



0.5 Interface Flask

A interface com o usuário foi desenvolvida utilizando o **framework Flask**, o que permitiu criar uma aplicação web funcional, integrando diretamente os módulos de **Indexador** e **RecuperacaoInformacao** em um ambiente via navegador. O código principal da interface reside em **front.py** e utiliza os templates **index.html** e **resultado.html**.

Estrutura e Inicialização

O fluxo da aplicação é configurado da seguinte forma:

1. Na inicialização do servidor (**front.py**), as instâncias do **Indexador** e do **RecuperacaoInformacao** são criadas, garantindo que o índice seja carregado do disco (**indice.txt**) ou construído a partir do corpus antes que o primeiro usuário realize uma busca.
2. A rota raiz (/) renderiza o template **index.html**, apresentando a interface de busca inicial.

Na interface de busca inicial, o usuário deve digitar a pesquisa na barra de busca e clicar no botão buscar

Processamento da Busca e Paginação

A rota `/buscar` é a parte mais importante da aplicação, tratando tanto novas consultas (POST) quanto a navegação de páginas (GET).

- **Requisição:** O termo de busca é extraído do formulário (POST) ou dos parâmetros da URL (GET).
- **Busca no Backend:** O método `busca.processar_consulta()` é invocado, que retorna a lista completa de `(doc_id, score)` ordenada por relevância (z-score) e filtrada pela lógica booleana.
- **Paginação:** Para melhorar a experiência do usuário, a lista completa de resultados é paginada.
 1. Uma constante de `RESULTADOS_POR_PAGINA` (definida como 10) é utilizada para determinar a quantidade mostrada por página.
 2. O número total de páginas é calculado, e a lista de resultados é dividida com base na página atual, para obter apenas os 10 a serem exibidos.

Geração de Snippets e Destaque

Para contextualizar cada resultado, a função `gerar_snippet()` é responsável por analisar o texto do documento, buscar a primeira ocorrência de qualquer termo da consulta, extrair um trecho do texto de tamanho máximo de 80 caracteres para frente e 80 para trás da primeira aparição do termo buscado, utilizar tags `<b>` para destacar o termo buscado no snippet e adicionar reticências sinalizando se há mais texto antes ou depois do que foi exibido.

A decisão de exibir a primeira aparição de qualquer termo de consulta foi uma decisão baseada na usabilidade e performance do programa. Como os documentos já são apresentados em ordem decrescente de relevância (calculada pelo z-score), o objetivo do snippet é validar rapidamente a presença do termo, e não reavaliar a relevância.

O template `resultado.html` recebe a lista formatada e aplica a lógica de front-end para renderizar os links de paginação (Anterior, números de página e Próxima), garantindo que o usuário possa navegar pelo conjunto completo de resultados ranqueados.

A notícia completa, pode ser visualizada ao clicar no título do resultado exibido. ela carregará em uma página html separada (`noticia.html`) com o conteúdo completo, a notícia da bbc referente ao resultado e o score de relevância daquela pesquisa.

Considerações Finais

As principais decisões de projeto buscaram equilibrar **simplicidade, desempenho e clareza**. A trie compacta otimizou o armazenamento; o índice persistente assegurou reuso e transparência; e o módulo de recuperação proporcionou consultas flexíveis e interpretáveis. A inclusão de operadores booleanos, o tratamento de erros sintáticos (como consultas malformadas) e o uso de OR implícito entre termos garantiram uma experiência de uso fluida e robusta. Essa arquitetura modular permite expansão futura, como por exemplo a inclusão do operador NOT.